

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Machine Learning (CO3117)

Báo Cáo HPMR

Hidden Markov Model

Giáo viên hướng dẫn: TS. Lê Thành Sách

Sinh viên thực hiện: Nguyễn Việt Hoàng 2311066

Nguyễn Chí Thanh 2313078

Lê Đức Tài 2312995

Mai Xuân Nhựt 2312549

THÀNH PHỐ HỒ CHÍ MINH, 05/09/2025

Mục lục

1	Tổng quan	4
2	Hidden Markov Model (HMM)	4
2.1	Định nghĩa HMM	4
2.2	Giải thuật Forward	5
2.3	Giải thuật Backward	6
2.4	Giải thuật Viterbi	7
2.5	Giải thuật huấn luyện tham số Baum-Welch	9

1 Tổng quan

Mô hình ẩn Markov (Hidden Markov Model - HMM) là một mô hình thống kê được sử dụng để mô tả các hệ thống mà trạng thái bên trong không thể quan sát trực tiếp, nhưng có thể suy ra thông qua các quan sát bên ngoài. HMM thường được áp dụng trong các lĩnh vực như nhận dạng giọng nói, xử lý ngôn ngữ tự nhiên, sinh học tính toán và nhiều ứng dụng khác.

Trong bài báo cáo này, nhóm sẽ trình bày về các khái niệm cơ bản của HMM, cách xây dựng và huấn luyện mô hình, cũng như các ứng dụng thực tế của nó. Nhóm sẽ sử dụng một tập dữ liệu cụ thể để minh họa quá trình huấn luyện và đánh giá mô hình HMM.

Tập dữ liệu này là bản thu âm các chữ số từ 0 đến 9, được thu thập từ nhiều người nói khác nhau. Mỗi bản ghi âm sẽ được chuyển đổi thành các đặc trưng âm thanh (features) để làm đầu vào cho mô hình HMM. Mục tiêu của chúng tôi là xây dựng một mô hình HMM có khả năng nhận diện chính xác các chữ số dựa trên các đặc trưng này.

2 Hidden Markov Model (HMM)

2.1 Định nghĩa HMM

Về mặt toán học, một Mô hình Markov Ẩn λ là một mô hình sinh (generative model) được định nghĩa bởi một bộ 5 thành phần $\lambda = (Q, V, A, B, \pi)$.

- $Q = \{q_1, q_2, \dots, q_N\}$: Một tập hợp hữu hạn N các **trạng thái ẩn**. Đây là các trạng thái bên trong của hệ thống mà chúng ta không thể quan sát.
- $V = \{v_1, v_2, \dots, v_M\}$ (đôi khi được ký hiệu là σ): Một tập hợp hữu hạn M các **ký hiệu quan sát** (observation symbols) hoặc “phát xạ”. Đây là những gì chúng ta có thể nhìn thấy.
- $A = \{a_{ij}\}$: **Ma trận xác suất chuyển trạng thái** (State Transition Probability Matrix), kích thước $N \times N$.
 - $a_{ij} = P(z_{t+1} = q_j | z_t = q_i)$, với z_t là trạng thái ẩn tại thời điểm t .
 - Đây là xác suất chuyển từ trạng thái q_i sang trạng thái q_j ở bước thời gian tiếp theo.
 - Ràng buộc: $\sum_{j=1}^N a_{ij} = 1$ cho mọi i .

- $B = \{b_j(k)\}$: **Ma trận xác suất phát xạ** (Emission Probability Matrix), kích thước $N \times M$.
 - $b_j(k) = P(x_t = v_k | z_t = q_j)$, với x_t là quan sát tại thời điểm t .
 - Đây là xác suất quan sát được ký hiệu v_k khi (given) hệ thống đang ở trạng thái ẩn q_j .
 - Ràng buộc: $\sum_{k=1}^M b_j(k) = 1$ cho mọi j .
- $\pi = \{\pi_i\}$: **Phân phối xác suất trạng thái ban đầu** (Initial State Probability Distribution), một vector kích thước N .
 - $\pi_i = P(z_1 = q_i)$.
 - Đây là xác suất hệ thống bắt đầu ở trạng thái q_i tại thời điểm $t = 1$.
 - Ràng buộc: $\sum_{i=1}^N \pi_i = 1$.

2.2 Giải thuật Forward

Giải thuật Forward là một phương pháp động (dynamic programming) được sử dụng để tính toán xác suất của một chuỗi quan sát nhất định $P(X|\lambda)$ trong HMM.

Hiện thực:

Tại vì tính toán xác suất nên số liệu rất nhỏ khi qua các bước sẽ gây nên underflow. Do vậy nên code giải thuật sẽ được tính dưới dạng log.

Bước 1: Khởi động - Tính $\alpha_1(i)$, với $i = 1, 2, \dots, N$.

$$\alpha_1(i) = \pi_i b_i(O_1) \quad (2.2)$$

Với hàm $\log_B(O)$ sẽ trả về giá trị xác suất cho 1 cột tương ứng với quan sát O .

```
log_alpha[:, 0] = self.logpi + self.log_B(O[0])
```

Bước 2: Đề quy - Tính $\alpha_t(i)$, với $t = 2, 3, \dots, T$ và $i = 1, 2, \dots, N$.

$$\alpha_t(i) = \sum_{k=1}^N [\alpha_{t-1}(k) a_{ki}] b_i(O_t) \quad (2.3)$$

Mở rộng 1 cột t của ma trận log alpha $1 \times N \rightarrow N \times N$ qua phép mở rộng chiều và phép cộng để cho các cột của ma trận bằng nhau.

```
for t in range(1, T):
    e = self.log_B(O[t])
    tmp = log_alpha[:, t-1][:, None] + self.logA
    log_alpha[:, t] = logsumexp(tmp, axis=0) + e
```

Với hàm logsumexp giúp tính log tổng của các số.

```
def logsumexp(x, axis=None):
    m = np.max(x, axis=axis, keepdims=True)
    s = m + np.log(np.sum(np.exp(x - m), axis=axis, keepdims=True))
    return np.squeeze(s, axis=axis)
```

Bước 3: Kết thúc - Tính $P(O|\lambda)$

$$P(O|\lambda) = \sum_{i=1}^N \alpha_T(i) \quad (2.4)$$

```
log_prob = logsumexp(log_alpha[:, T-1])
```

Độ phức tạp: $O(N^2T)$

2.3 Giải thuật Backward

Hiện thực:

Tương tự như giải thuật Forward, ta sẽ tính dưới dạng log để tránh underflow.

Bước 1: Khởi động - $\beta_T(i) = 1$, với $i = 1, 2, \dots, N$.

```
logBeta[:, T-1] = 0.0
```

Bước 2: Đệ quy - Tính $\beta_t(i)$ với

- $t = T - 1, T - 2, \dots, 1$
- $i = 1, 2, \dots, N$

$$\beta_t(i) = \sum_{k=1}^N [a_{ik} b_k(O_{t+1}) \beta_{t+1}(k)] \quad (2.1)$$

Tương như cách tính của Forward

```
for t in range(T-2, -1, -1):
    e_next = self.log_B(O[t+1])
    tmp = self.logA + e_next[None, :] + logBeta[:, t+1][None, :]
    logBeta[:, t] = logsumexp(tmp, axis=1)
```

Độ phức tạp: $O(N^2T)$

2.4 Giải thuật Viterbi

Hiện thực:

Tương tự như giải thuật Forward, ta sẽ tính dưới dạng log để tránh underflow.

Bước 1: Khởi động - Tính $\delta_1(i)$ và $\varphi_1(i)$ theo công thức (3.1) và (3.2), với $i = 1, 2, \dots, N$.

$$\delta_1(i) = \pi_i b_i(O_1) \quad (3.1)$$

$$\varphi_1(i) = 0 \quad (3.2)$$

```

delta = np.full((self.N, T), -np.inf)
phi = np.zeros((self.N, T), dtype=np.int32)
delta[:, 0] = self.logpi + self.log_B(O[0])

```

Bước 2: Đệ quy - Với $t = 2, \dots, T$ và $i = 1, \dots, N$, cập nhật $\delta_t(i)$ và $\phi_t(i)$ theo công thức (3.3) và (3.4).

$$\delta_t(i) = \max_{1 \leq k \leq N} [\delta_{t-1}(k) a_{ki}] b_i(O_t) \quad (3.3)$$

$$\phi_t(i) = \operatorname{argmax}_{1 \leq k \leq N} [\delta_{t-1}(k) a_{ki}] \quad (3.4)$$

Ta sẽ tính như forward và lấy max theo cột.

```

for t in range(1, T):
    e = self.log_B(O[t])
    scores = delta[:, t-1][:, None] + self.logA
    phi[:, t] = np.argmax(scores, axis=0)
    delta[:, t] = np.max(scores, axis=0) + e

```

Bước 3: Kết thúc - Xác định trạng thái tối ưu tại thời điểm T và xác suất tương ứng, dùng công thức (3.5) và (3.6).

$$P^* = \max_{1 \leq k \leq N} \delta_T(k) \quad (3.5)$$

$$q_T^* = \operatorname{argmax}_{1 \leq k \leq N} \delta_T(k) \quad (3.6)$$

```

path = np.zeros(T, dtype=np.int32)
path[T-1] = np.argmax(delta[:, T-1])
log_path_prob = np.max(delta[:, -1])

```


Bước 4: Truy vết ngược - Xác định dãy trạng thái tối ưu dùng Bảng $\varphi_t(i)$ và công thức (3.7). Với $t = T - 1, T - 2, \dots, 1$

$$q_t^* = \varphi_{t+1}(q_{t+1}^*) \quad (3.7)$$

```
for t in range(T-2, -1, -1):  
    path[t] = phi[path[t+1], t+1]
```

Độ phức tạp: $O(N^2T)$

2.5 Giải thuật huấn luyện tham số Baum-Welch

Đối với dạng quan sát, ta sẽ có cách hiện thực khác nhau. Do vậy nhóm sẽ hiện thực HMM dưới dạng 1 class:

class BaseHMM

- **Phương thức khởi tạo** - Nhận vào số trạng thái ẩn N , số quan sát M và khởi tạo các tham số π, A, B ngẫu nhiên.
- **Phương thức log_B(O)** - Nhận vào một quan sát O và trả về xác suất phát xạ dưới dạng log.
- **Phương thức forward(O)** - Thực hiện giải thuật Forward và trả về log xác suất của chuỗi quan sát O .
- **Phương thức backward(O)** - Thực hiện giải thuật Backward và trả về ma trận log beta.
- **Phương thức viterbi(O)** - Thực hiện giải thuật Viterbi và trả về dãy trạng thái tối ưu cùng log xác suất tương ứng.
- **fit(self, data, n_loop=50, bound_learning=1e-4)** - Thực hiện giải thuật Baum-Welch để huấn luyện tham số mô hình dựa trên tập train D .

Hiện thực class này sẽ không cố định việc khởi tạo các ma trận tham số ban đầu là ngẫu nhiên mà sẽ nhận vào các ma trận để người dùng có thể tự config việc sử dụng.

Phân phối xác suất của quan sát O thường có 2 dạng: Catagorical và Gaussian. Do vậy nhóm sẽ kế thừa class BaseHMM để hiện thực 2 class con. Hầu như khác biệt giữa 2 dạng là ở hàm logB và fit vì vậy nên các phương thức còn lại sẽ hiện thực ở lớp basic.

Giải thuật Baum-Welch (EM Algorithm)

Giải thuật Baum-Welch là một thuật toán EM (Expectation-Maximization) được sử dụng để ước lượng các tham số của HMM khi chỉ có chuỗi quan sát O mà không biết chuỗi trạng thái ẩn.

Bước 1: Khởi động - Gán giá trị ngẫu nhiên cho các thông số cần học π , A , B . Lưu ý, từng hàng của ma trận A , B và vector π phải là các phân phối xác suất.

Lặp lại E-step và M-step đến khi hội tụ:

Bước 2: Thực hiện bước tính kỳ vọng (E-step):

- Tính $\alpha_t(i)$ và $\beta_t(i)$ dùng giải thuật lan truyền thuận và ngược ở trên.

Hàm logB

Đối với **discreteHMM**

Trả về 1 cột tương ứng quan sát o_t của ma trận logB.

```
def log_B(self, o_t) -> np.ndarray:  
    return self.logB[:, o_t]
```

Đối với **continueHMM**

Trả về cả 1 ma trận log xác suất của cả chuỗi quan sát O theo 2 tham số mean và covar của mô hình hiện tại.

Xác suất của một quan sát O_t cho D chiều theo phân phối Gaussian đa biến được tính theo công thức:

$$p(O_t|\mu, \sigma) = \frac{1}{(2\pi)^{D/2}|\sigma|^{1/2}} \exp\left(-\frac{1}{2}(O_t - \mu)^T \sigma^{-1}(O_t - \mu)\right)$$

Lấy log:

$$\log p(O_t|\mu, \sigma) = -\frac{D}{2}\log(2\pi) - \frac{1}{2}\log|\sigma| - \frac{1}{2}(O_t - \mu)^T \sigma^{-1} (O_t - \mu)$$

Ta có:

- self._precisions là ma trận nghịch đảo của covar
- np.einsum('ntd,ndk,ntk->nt', diff, self._precisions, diff)

Tính toán $\text{quad}[i, t] = (O_t - \mu_i)^T \Sigma_i^{-1} (O_t - \mu_i)$

- $\log_norm_consts[i] = -\frac{D}{2}\log(2\pi) - \frac{1}{2}\log|\sigma_i|$.

```
def log_B_sequence(self, O: np.ndarray) -> np.ndarray:
    T = O.shape[0]
    diff = O[None, :, :] - self.means[:, None, :]
    quad = np.einsum('ntd,ndk,ntk->nt', diff, self._precisions,
                    diff)
    return self._log_norm_consts[:, None] - 0.5 * quad
```

Tính toán forward và backward:

Tận dụng lại ma trận xác suất phát xạ đã tính ở bước trên để tiết kiệm thời gian tính toán.

```
log_prob, log_alpha, log_b_all = self.forward(O)
logBeta = self.backward(O, log_b_all)
```

- Tính $\xi_t(i, j)$ và $\gamma_t(i)$ theo công thức (4.3) và (4.2).

Biến xi - Xác suất chuyển trạng thái:

Biến $\xi_t(i, j)$ biểu thị xác suất hệ thống ở trạng thái S_i tại thời điểm t và chuyển sang trạng thái S_j tại thời điểm $t + 1$:

$$\xi_t(i, j) = P(q_t = S_i, q_{t+1} = S_j | O, \lambda) \quad (2.2)$$

$$\xi_t(i, j) = \frac{\alpha_t(i) a_{ij} b_j(O_{t+1}) \beta_{t+1}(j)}{P(O|\lambda)} \quad (4.3)$$

Trong log-domain:

```
# log_xi: (N, N, T-1)
# tmp[i,j,t] = log_alpha[i,t] + logA[i,j] + log_b[j,t+1] + logBeta[j,t+1]
tmp = (log_alpha[:, :-1].reshape(self.N, 1, T-1) # (N, 1, T-1)
      + self.logA[:, :, None]                    # (N, N, 1)
      + log_b_all[None, :, 1:]                    # (1, N, T-1)
      + logBeta[None, :, 1:])                     # (1, N, T-1)
log_xi = tmp - log_prob
xi = np.exp(log_xi) # (N, N, T-1)
```

Biến gamma - Xác suất trạng thái:

Biến $\gamma_t(i)$ biểu thị xác suất hệ thống ở trạng thái S_i tại thời điểm t , cho trước chuỗi quan sát O và mô hình λ :

$$\gamma_t(i) = P(q_t = S_i | O, \lambda) = \frac{\alpha_t(i)\beta_t(i)}{\sum_{i=1}^N \alpha_t(i)\beta_t(i)} \quad (4.2)$$

Trong log-domain:

```
log_gamma = log_alpha + logBeta - log_prob
gamma = np.exp(log_gamma) # (N, T)
```

Bước 3: Thực hiện bước cực đại hóa kỳ vọng (M-step): Tính π , A , B dùng các công thức (4.5), (4.6), và (4.7).

Cập nhật phân phối ban đầu π :

$$\pi_i = \gamma_1(i) \quad (4.5)$$

```
pi_numerator += gamma[:, 0]
# M-step
self.pi = pi_numerator / len(data)
```

Cập nhật ma trận chuyển trạng thái A :

$$a_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)} \quad (4.6)$$

```
A_numerator += xi.sum(axis=2)
A_denominator += gamma[:, :-1].sum(axis=1, keepdims=True)
# M-step
self.A = A_numerator / (A_denominator + 1e-12)
```

Cập nhật ma trận phát xạ B :

Đối với **discreteHMM** (quan sát rời rạc):

$$b_j(k) = \frac{\sum_{t=1, O_t=v_k}^T \gamma_t(j)}{\sum_{t=1}^T \gamma_t(j)} \quad (4.7)$$

```
B_denominator += gamma.sum(axis=1, keepdims=True)
O_matrix = np.zeros((T, M))
O_matrix[np.arange(T), O] = 1.0 # O_matrix[t, O[t]] = 1
B_numerator += gamma @ O_matrix
# M-step
self.B = B_numerator / (B_denominator + 1e-12)
```

Đối với **continueHMM** (quan sát liên tục - Gaussian):

$$\mu_j = \frac{\sum_{t=1}^T \gamma_t(j) O_t}{\sum_{t=1}^T \gamma_t(j)} \quad (4.8)$$

$$\sigma_j = \frac{\sum_{t=1}^T \gamma_t(j) (O_t - \mu_j)(O_t - \mu_j)^T}{\sum_{t=1}^T \gamma_t(j)} \quad (4.9)$$

```

gamma_sum += gamma.sum(axis=1, keepdims=True)
means_numerator += gamma @ 0 # (N,T) @ (T,D)

# Covariance accumulation using einsum
cov_numerator += np.einsum('nt,td,tk->ndk', gamma, 0, 0)

# M-step: update means and covariances
self.means = np.nan_to_num(means_numerator / (gamma_sum + 1e-12))
S2_term = np.nan_to_num(cov_numerator / (gamma_sum[:, :, None] + 1e-12))
mean_outers = self.means[:, :, None] @ self.means[:, None, :]
self.covariances = S2_term - mean_outers

```

Điều kiện dừng: Thuật toán lặp lại cho đến khi log-likelihood hội tụ:

$$|LL_{iter} - LL_{iter-1}| < \varepsilon \quad (2.3)$$

Độ phức tạp: $O(N^2T \cdot K)$ với K là số vòng lặp EM.

Tài liệu tham khảo

